Abbreviated title: A3E toolbox

OpenDvM: Decoding the Visual Mind — A Unified Toolbox for EEG/MEG Attention Research

Dirk van Moorselaar[1,], Sebastiaan Mathôt[2,] & Stefan Van der Stigchel[1]

1. Faculty of Social and Behavioral Sciences, Experimental Psychology, Helmholtz Institute, Utrecht University, Utrecht, the Netherlands
2. Department of Psychology, University of Groningen, the Netherlands

Conflict of Interest: The authors declare no competing financial interests

Number of pages:
Number of Figures:
Number of Tables:

Acknowledgements

This research was supported by a NWO VICI grant to SVdS.

Please address correspondence to:
Dr. Dirk van Moorselaar
Faculty of Social and Behavioral Sciences, Experimental Psychology
Utrecht University, Utrecht
e-mail: dirkvanmoorselaar@gmail.com

1. Introduction

Understanding how attentional selection unfolds over time is a central challenge in cognitive neuroscience. Among the various neuroimaging methods available, electroencephalography (EEG) is uniquely suited for this purpose due to its high temporal resolution, enabling researchers to track the rapid dynamics of neural processing and attentional selection on a millisecond timescale (Cohen, 2014; Hillyard & Anllo-Vento, 1998; Luck, 2014).

Historically, EEG-based attention research has primarily relied on event-related potentials (ERPs) to identify stimulus-locked modulations of neural activity. These univariate analyses have been instrumental in revealing the temporal structure of attentional processes (Eimer, 1996; Luck & Hillyard, 1994a). However, in recent years, there has been a growing interest in multivariate approaches—such as decoding and encoding models—which offer complementary insights by capturing distributed patterns of activity across electrodes (Fahrenfort et al., 2018; Grootswagers et al., 2017; King & Dehaene, 2014). In attention research, which has a rich history of well-established univariate findings (Hillyard & Anllo-Vento, 1998; Luck, 2012), combining multivariate with traditional approaches has particular potential to provide deeper characterization of attentional mechanisms. These multivariate methods can reveal the representational content of neural signals, detect subtle attentional shifts that may not be evident in traditional ERP waveforms, and quantify experimental effects without a-priori electrode selection, often detecting condition differences that are harder to identify in conventional analyses (Fahrenfort et al., 2017; Fahrenfort et al., 2018).

While powerful analytical toolboxes such as MNE-Python (Gramfort et al., 2013), FieldTrip (Oostenveld et al., 2011), EEGLAB (Delorme & Makeig, 2004), and scikit-learn (Pedregosa et al., 2011) provide the necessary functionality for these diverse techniques, integrating them into cohesive analysis workflows requires navigating distinct syntaxes, data structures, and implementation details across packages. For researchers conducting attention studies that combine multiple analysis types—comparing ERP components with decoding time courses, or relating time-frequency modulations to forward encoding reconstructions—maintaining consistency in data handling, preprocessing, and output formatting across these different analytical approaches can be time-consuming and error-prone.

To streamline these analytical workflows, we developed the **OpenDvM**, an open-source Python framework that provides a unified interface for ERP analysis, time-frequency decomposition, multivariate decoding, and forward encoding models. OpenDvM is designed for researchers who understand these analytical concepts but seek more efficient

implementation. The toolbox enforces consistent syntax and data handling across all analysis types, automates technical details like trial balancing and cross-validation, and facilitates integration between univariate and multivariate approaches. Building on the robust foundations of MNE-Python and scikit-learn (Gramfort et al., 2013; Pedregosa et al., 2011), OpenDvM provides a specialized layer optimized for attention research workflows. Notably, OpenDvM implements channel tuning function (CTF) analysis (Foster et al., 2016, 2017) in Python, enabling researchers to reconstruct continuous spatial representations without requiring MATLAB-based toolboxes.

The toolbox has been developed and refined through application to diverse attention research projects, including investigations of learned distractor suppression (van Moorselaar et al., 2018; van Moorselaar et al., 2020), expectation-driven attentional modulation (Duncan, Theeuwes, et al., 2023; van Moorselaar et al., 2023), spatial working memory mechanisms (van Moorselaar et al., 2018), and neural representations of predicted events (Duncan, van Moorselaar, et al., 2023; Huang et al., 2025; Li et al., 2024). These studies span all four analysis types implemented in OpenDvM - ERP (Duncan, Theeuwes, et al., 2023; Oxner et al., 2025; van Moorselaar et al., 2023; van Moorselaar, Wang, et al., 2025), time-frequency decomposition (Duncan, Theeuwes, et al., 2023; van Moorselaar et al., 2021), multivariate decoding (Alilović et al., 2021; Duncan, van Moorselaar, et al., 2023; Li et al., 2024; van Moorselaar et al., 2020; van Moorselaar & Slagter, 2019) and forward encoding (Huang et al., 2025; van Moorselaar et al., 2018; van Moorselaar & Slagter, 2019; van Moorselaar, Theeuwes, et al., 2025) - demonstrating the toolbox's practical utility across different experimental paradigms and research questions.

To illustrate OpenDvM 's core functionality, we analyze data from an EEG experiment investigating multiple aspects of attentional selection. We selected this unpublished dataset specifically because its design encompasses diverse attentional mechanisms (feature-based selection, spatial attention, distractor suppression), enabling comprehensive demonstration of OpenDvM 's analytical range. While individual published studies typically focus on targeted experimental manipulations, this paradigm's versatility allows us to illustrate how different analysis types can be applied within a unified framework, highlighting the toolbox's integrative capabilities.

2. Materials and methods

2.1 Participants

Data used for this demonstration were collected from 7 participants and selected based on data quality criteria including low artifact levels, adequate task performance, and clear EEG signal quality. The final dataset comprises 5 participants (age range: X-X years, X female) selected through visual inspection of data quality and task performance. All participants provided written informed consent and the study was approved by the local ethics committee (details).

2.2 Task

The experimental paradigm consists of two phases designed to demonstrate multiple aspects of attentional selection (Figure 1). Each trial began with a fixation period (600-800 ms, randomly jittered), followed by a search display containing 8 shapes (diamonds or circles, grey line drawings) arranged in a circular configuration around fixation. Search display were presented until response with a timeout of 1500 ms..

In the main experimental phase, trials were organized in a 2×2 factorial design with four equally probable conditions: target-present/distractor-present, target-absent/distractor-present, target-present/distractor-absent, and target-absent/distractor-absent. On target-present trials, one homogeneous shape was replaced by a unique shape (circle among diamonds or vice versa). On distractor-present trials, one shape was replaced by a colored image selected from four categories (car, dog, guitar, house). Critically, targets and distractors appeared only at cardinal axis positions, and when present, distractors appeared with higher probability (50%) at one specific location, creating a spatial attentional bias. Participants reported via button press whether a unique shape was present or absent.

Prior to the experimental phase, participants completed a training phase using target-absent/distractor-present displays. One distractor image was presented for 250 ms, and participants indicated via button press whether it matched the image from the preceding trial. This training phase provides an independent dataset for multivariate decoding and encoding analyses. The paradigm includes [total number] trials per participant ([breakdown by condition]) and engages multiple attentional mechanisms including feature-based attention (shape detection) and spatial attention (location probability bias), making it well-suited for demonstrating diverse analytical approaches.

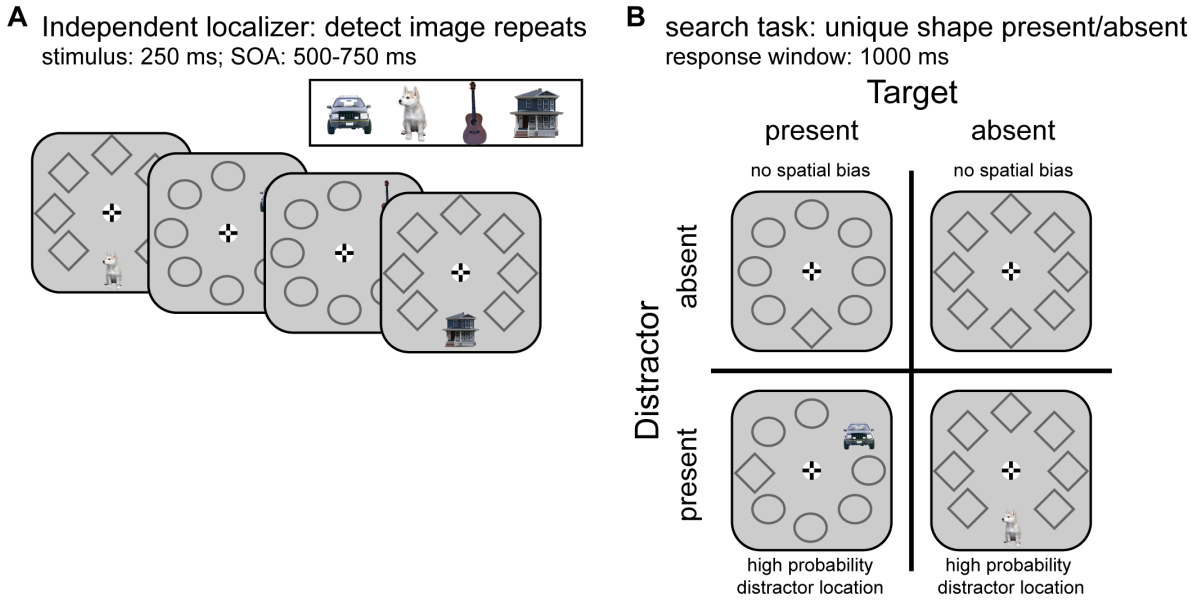


Figure 1 | Experimental paradigm. The experiment started with a separate training phase, which served as an independent localizer for multivariate analyses followed by the main task. Per display, participants had to indicate whether the display contained a unique shape (diamond among circles or vice versa). Search displays could also contain colored images, which when present, appeared with a higher probability at a particular location in space.

2.3 Data acquisition and organization

EEG data were recorded at 512 Hz using a Biosemi ActiveTwo system with 32 electrodes positioned according to the 10-10 system and reference electrodes at the left and right earlobes. Vertical and horizontal EOG were recorded via external electrodes placed ~2 cm above and below the eye and ~1 cm lateral to the external canthi. Eye movements were simultaneously recorded using an Eyelink 1000 (SR Research) eye tracker sampling the left eye at 1000 Hz. Behavioral data and task parameters were recorded using OpenSesame (Mathôt et al., 2012). All raw and preprocessed data are publicly available via the Open Science Framework (OSF) [insert link] and are structured according to OpenDvM's required folder and naming conventions (see Figure X; section 3.2). This data organization ensures direct compatibility with the toolbox and supports streamlined preprocessing and analysis workflows.

3. Toolbox specifics

3.1 Requirements

Paragraph with python installation, etc. Ask Sebastiaan

3.2 Data organization and Folder structure

OpenDvM is designed to work primarily with preprocessed EEG/MEG data, allowing users to apply the toolbox's analysis functions (ERP, time-frequency, decoding, encoding; consider adding figure) to their existing processed datasets. However, OpenDvM also includes optional preprocessing functionality for users who wish to process raw data within the same framework. The toolbox requires a standardized folder structure to ensure consistent data handling and seamless integration between different data modalities, enabling efficient data location and processing across all analysis types.

The expected folder structure organizes data into separate directories for each modality (Listing 1). EEG data should be organized with raw files in `eeg/raw/` and processed files in `eeg/processed/`. MEG data follows an identical structure in `meg/raw/`and `meg/processed/`. Behavioral data files should be placed in `behavioral/raw/`, and eye-tracking files (if available) in `eye/raw/`. If you already have preprocessed data, you can start directly with files in `eeg/processed/` or `meg/processed/` and skip the raw data folders entirely.

Listing 1 | Example folder structure for project organization.

```
project_directory/
├── eeg/
│   ├── raw/          # Raw EEG files (optional; used for preprocessing)
│   └── processed/    # Preprocessed EEG files (required for all analyses)
├── beh/
│   ├── raw/          # Behavioral data files (optional)
│   └── processed/    # Processed behavioral files (optional)
└── eye/
    ├── raw/          # Eye-tracking data files (optional)
    └── processed/    # Processed eye-tracking files (optional)
```

Behavioral data can be stored in two formats: either linked to M/EEG epochs using MNE metadata, or as separate CSV files in `behavioral/processed/`. For preprocessed data imported from external sources, the metadata approach is often most convenient. Eye-tracking data, when available, should be stored as .npz files in `eye/processed/` containing x and y coordinates per epoch, time samples, and sampling frequency.

All OpenDvM analysis functions require preprocessed, epoched data stored in `eeg/processed/` (or `meg/processed/`) using the naming

convention `sub_[ID]_all_[pipeline]-epo.fif` (e.g., `sub_01_all_main-epo.fif` for concatenated sessions) or `sub_[ID]_ses_[NUM]_[pipeline]-epo.fif` for individual sessions. OpenDvM enforces two-digit zero-padded subject and session IDs (e.g., `sub_01`, `ses_01`) for reliable file discovery and sorting, supporting up to 99 subjects. Users can either generate these files using OpenDvM's preprocessing pipeline or place their own preprocessed data, converted to MNE-Python .fif format, directly in the appropriate folder.

For users utilizing the preprocessing functionality, raw EEG/MEG recordings should be stored in `eeg/raw/` (or `meg/raw/`) using the format `sub_[ID]_ses_[NUM].{bdf/edf/fif}` for single-run sessions or `sub_[ID]_ses_[NUM]_run_[NUM].{bdf/edf/fif}` for multi-run sessions (e.g., `sub_01_ses_01_run_01.bdf`). Multiple runs within the same session are automatically concatenated during preprocessing. Behavioral files should follow the same naming convention with a .csv extension: `sub_[ID]_ses_[NUM].csv` or `sub_[ID]_ses_[NUM]_run_[NUM].csv` when sessions contain multiple runs. Each row represents a single trial, with columns containing experimental parameters (stimulus type, condition, response time, accuracy) that align temporally with EEG/MEG triggers. Eye-tracking files, when available, follow the same naming structure in the `eye/raw/` directory.

This session-based organization enables independent preprocessing of each recording session before optional concatenation for analysis, supporting flexible experimental designs while maintaining data integrity across preprocessing and analysis stages.

OpenDvM is designed to work primarily with preprocessed EEG data, allowing users to apply the toolbox's analysis functions (ERP, time-frequency, decoding, encoding) to their existing processed datasets. However, OpenDvM also includes optional preprocessing functionality for users who wish to process raw data within the same framework. The toolbox requires a standardized folder structure to ensure consistent data handling and seamless integration between different data modalities, enabling efficient data location and processing across all analysis types (Figure 2).

The expected folder structure organizes data into separate directories for each modality (see Listing 1). EEG data should be organized with raw files in `eeg/raw/` and processed files in `eeg/processed/`. Behavioral data files should be placed in `beh/raw/`, and eye-tracking files (if available) in `eye/raw/`. If you already have preprocessed data, you can start directly with files in `eeg/processed/` and skip the raw data folders entirely. In the latter case, behavior needs to either be linked to M/EEG epochs using MNE metadata or stored as a sepearte file in `beh/processed/`. If available, x, y coordinates from an eyetracker can be stored as a .npz file in `eye/processed/`.

All OpenDvM analysis functions require preprocessed, epoched data stored in `eeg/processed/` using the naming convention `subject_[ID]_[pipeline]_epo.fif` (e.g., `subject_01_main_epo.fif`). Users can either generate these files using OpenDvM's preprocessing pipeline or place their own preprocessed data, converted to MNE-Python .fif format, directly in this folder. The pipeline identifier allows you to maintain multiple preprocessing versions of the same dataset.

For users utilizing the preprocessing functionality, raw EEG recordings should be stored in `eeg/raw/` using the format `subject_[ID]_session_[NUM]_run_[NUM]` (e.g., `subject_01_session_1_run_1.bdf`). This structure allows data from different sessions to be preprocessed independently, while multiple runs within the same session are automatically concatenated during preprocessing. Behavioral files should follow the same naming convention as raw EEG files but with a .csv extension: `subject_[ID]_session_[NUM]_run_[NUM].csv` or `subject_[ID]_session_[NUM].csv` when combining runs. Each row should represent a single trial or event, with columns containing relevant experimental parameters such as stimulus type, condition, response time, and accuracy that align temporally with EEG triggers. Eye-tracking files, when available, should use the same naming structure in the `eye/raw/` directory.

This organization supports two main workflows: users can either utilize the full pipeline from raw data through analysis, or focus exclusively on OpenDvM's analysis capabilities by working directly with preprocessed data. This flexibility accommodates researchers with existing preprocessing workflows while providing comprehensive functionality for those requiring end-to-end processing.

3.3 Preprocessing Pipeline Overview

A3E includes an optional, modular preprocessing pipeline built on top of the MNE-Python framework (Gramfort et al., 2013). This pipeline streamlines standard EEG preprocessing steps such as filtering, re-referencing, artifact rejection, and epoching, while also offering additional functionality.

Specifically, the pipeline includes tools for automated artifact detection, as well as functionality for aligning EEG recordings with raw behavioral data files. These enhancements aim to reduce manual intervention and ensure consistency across participants and sessions.

Importantly, the use of this preprocessing pipeline is not a requirement for using A3E. All analysis modules are compatible with any epoched EEG data in MNE's Epochs format. This flexibility allows users to integrate A3E into existing pipelines or adapt it for use with data preprocessed using other tools. Further details about the preprocessing functionality are provided in Section [X.X].

3.4 OpenDvM Architecture and Core Functionality

OpenDvM is designed to make advanced EEG analyses—such as ERPs, time–frequency decompositions, multivariate decoding, and forward encoding—accessible and reproducible within a unified, modular framework. The framework is unified in the sense that every analysis follows the same two-step workflow and consistent syntax, so users only need to learn the procedure once. At the same time, it is modular: each analysis type is encapsulated in its own class, which can be easily extended or combined with others without changing the overall structure of the toolbox.

Each single-subject analysis proceeds through two steps. First, the user initializes an analysis-specific class, providing preprocessed data and relevant parameters (e.g., a baseline correction window for ERP analysis, or the number of cross-validation folds for decoding). Second, the analysis is executed by calling a class-specific method, which produces the requested outputs.

All analysis classes support dynamic parameter updating, allowing users to modify preprocessing settings after initialization. Parameters can be updated by directly assigning new values to class attributes (e.g., `erp_o.baseline = (-0.3, 0)`), enabling efficient exploration of different parameter combinations without repeatedly loading and initializing data. This flexibility is particularly valuable when systematically testing preprocessing choices or when different analyses require different settings

This consistent initialization–execution pattern ensures that analyses are transparent for end users while remaining flexible for researchers developing new methods. Outputs are automatically saved in standardized subdirectories (`erp`, `tf`, `bdm`, `ctf`), ensuring seamless integration with group-level functions and plotting utilities. The following sections provide detailed demonstrations of each analysis type and its use cases.

3.5 Reproducibility and Workflow Automation

Reproducibility is a central design principle of A3E. To support transparent and repeatable research, A3E automatically saves key metadata and intermediate outputs at every stage of the analysis pipeline. This includes parameter settings, file versioning, and the structure of output directories, which are systematically maintained to ensure traceability.

The toolbox's consistent architecture and modular structure make it straightforward to automate workflows across participants or sessions. Once a pipeline is defined for a single subject, it can be easily applied across a study using standard scripting approaches, promoting consistency and scalability.

Together, these features reduce the risk of manual errors, promote consistency across studies, and facilitate the sharing and replication of results.

4. Single subject and group level analysis

All OpenDvM analyses operate on preprocessed, epoched EEG data stored in MNE-Python's `-epo.fif` format. At the single-subject level, data are loaded using the `load_processed_eeg()` function, which relies on MNE's standard I/O functions. Behavioral parameters can be stored either as metadata within the epoched file or as a separate CSV file with matching filename (specified via `csv_file=True`). The toolbox expects behavioral data as pandas DataFrames, with each row representing a single trial and columns containing experimental variables (stimulus properties, response information, trial type, etc.) aligned with the epoched EEG data.

OpenDvM also provides optional trial exclusion based on eye movement artifacts. Users can exclude trials containing gaze deviations exceeding specified thresholds during critical analysis windows, accommodating both dedicated eye-tracking data (stored as .npz files) and EOG channels within the EEG recording. Exclusion parameters—including the temporal window for monitoring eye movements, angular deviation thresholds, screen

parameters, and optional drift correction—are specified via an `eye_dict` dictionary passed to the loading function (Listing 2).

Listing 2 | Reading in preprocessed data using load_processed_epochs in the OpenDvM toolbox.

```
# parameters to exclude trials based on eye movement behavior
eye_dict = {
      'use_tracker': True,                      # use eye tracker data
      'window_oi':(0,0.3),                      # eye movemnt window
      'angle_thresh':1,                         # threshold (°visual angle)
      'viewing_dist': 70,                       # viewing distance (cm)
      'screen_res': (1920, 1080),               # screen resolution (pixel)
      'screen_h': 29,                           # screen heigt (cm)
      'drift_correct':(-0.2,0)}                 # drift correction window

# read in preprocessed epochs and exclude
df, epochs = FolderStructure().load_processed_epochs(
      sj = 1,                                   # subject identifier
      fname = 'ses_1_main',                     # file name
      preproc_name = 'main',                    # preprocessing name
      eye_dict = eye_dict                       # exclude eye
      csv_file = False)                         # read in metatdata
```

The general workflow involves initializing an analysis class with the loaded data and relevant parameters, then executing specific analysis methods. Results are automatically saved to standardized output directories organized by analysis type (`erp/`, `tfr/`, `bdm/`, `ctf/`), supporting subsequent group-level analyses where dedicated functions aggregate single-subject results for visualization and statistical inference.

The following subsections detail the specific functionality and parameters for each analysis type, demonstrating how OpenDvM's unified framework applies to event-related potentials (ERP), time-frequency analysis (TFR), multivariate decoding (BDM), and forward encoding models (CTF). While examples focus on attention research paradigms, the modular design accommodates diverse experimental questions and cognitive domains.

4.1 ERP Analysis

4.1.1 ERPs and lateralized difference waveforms

ERPs have played a central role in EEG research on attention, with numerous components linked to distinct stages of attentional selection and processing (Eimer, 1996; Luck & Hillyard, 1994a). Among these, *lateralized* components—such as the N2pc—have been especially influential in visual attention research. The N2pc (N2-posterior-contralateral) reflects the difference in neural activity over posterior electrode sites contralateral versus ipsilateral to

attended visual targets, providing a precise temporal marker of spatially selective attention that typically emerges 180-300 ms after stimulus onset (Eimer, 1996; Luck & Hillyard, 1994b; Woodman & Luck, 2003). This component has been widely replicated across visual search paradigms and offers unique insights into the dynamics of attentional selection (Lien et al., 2008; Mazza et al., 2007). To demonstrate the ERP functionality of OpenDvM, we focus on lateralized ERP analyses using the N2pc as a case study, noting that the same logic can be easily applied to create non-lateralized, condition-specific ERPs.

4.1.2 ERP specific parameters (single subject level)

The ERP class accepts several optional parameters that control preprocessing steps applied during initialization. Beyond the required inputs (subject identifier, EEG data as an MNE `Epochs` object, and behavioral data as a pandas `DataFrame`), users can specify baseline correction windows, filter settings, and downsampling parameters to optimize data for ERP analysis. In the example in Listing 3, the ERP instance applies a 30 Hz low-pass filter to reduce high-frequency artifacts and baseline-corrects each epoch using the 100 ms pre-stimulus interval.

Listing 3 | Initializing an ERP object with baseline correction and filtering in the OpenDvM Toolbox.

```
# Initialize ERP class with filtering and baseline correction
erp_o = ERP(sj, epochs, df,
      baseline=(-0.1, 0),     # 100ms pre-stimulus baseline
      l_filter=None,          # no high-pass filter
      h_filter=30,            # 30Hz low-pass filter
      downsample=None)        # no downsampling
```

4.1.3 Lateralized difference waveforms (single subject level)

Computing lateralized difference waveforms, such as the N2pc, requires careful selection and alignment of trials to isolate hemispheric asymmetries in response to lateralized stimuli (e.g., targets or distractors). The OpenDvM function `condition_erps()` streamlines this process by providing flexible options that match established practices in the literature.

The function enables users to select relevant target positions (`pos_labels`), restrict analyses to trials with vertical distractors (`midline`), and align hemispheric activity via topographic flipping (`topo_flip`). The restriction to vertical midline distractors is particularly important, as it ensures that resulting lateralization reflects target-specific processes rather than activity driven by lateralized distractors—a well-established practice in N2pc research(Hickey

et al., 2009). Additional parameters allow for condition-specific analyses (`cnds`), temporal windowing (`time_oi`), and trial exclusion criteria (`excl_factor`).

Listing 4 | Computing lateralized ERPs (N2pc) with condition_erps() in the OPENDvM Toolbox.

```
# create lateralized difference waveform yoked to targets
erp_o.condition_erps(cnds = None,          # collapse across conditions
      pos_labels=dict(target_loc=[2,6]),  # limit to left/right targets
      midline= dict(dist_loc = [0,4]),    # limit to vertical distractors
      topo_flip=dict(target_loc=[2]),     # flip left target topography
      time_oi = (-0.1,0.4),               # analysis time window
      excl_factor = dict(block_type = ['loc']),# exclude train trials
      name = 'n2pc_main_task')            # output file name
```

A complete example is shown in Listing 4. This call computes target-locked lateralized ERPs, excludes trials with lateralized distractors, and aligns hemifield activity to yield a clean N2pc waveform. The resulting data are stored under the name `'n2pc_main_task'` for visualization or group-level analysis.

4.1.4 Report Generation for Quality Control

OpenDvM offers optional quality control report generation using the `mne.Report` framework, which can be enabled by setting `report=True` during class initialization. These comprehensive reports include topographical maps and time course plots for all electrodes across each condition of interest, allowing users to visually inspect evoked responses and identify potential data quality issues. Quality control reports are particularly valuable for detecting atypical patterns or anomalies at the subject level, enabling informed decisions about data inclusion before proceeding to group-level analyses. The reports are saved as standalone HTML files within the automatically generated `erp/` folder and can be opened for interactive inspection or generated automatically across multiple subjects in batch processing workflows.

4.1.5 Visualization and Statistical Analysis (group level)

Group-level analyses require aggregating single-subject results stored in the `erp/` folder. OpenDvM provides flexible functionality to load and combine data across subjects, with options to specify subject subsets or restrict analyses to particular experimental conditions. By default, data from all available subjects are loaded, but users can specify a subset of participants or filter for specific experimental contrasts. The code snippet in Listing 5 demonstrates loading the N2pc waveform data across all subjects. Once the data are aggregated, group-level statistics

and visualization can be performed using the corresponding functions described in the following sections.

Listing 5 | Reading aggregated ERP data with read_erps() in the OpenDvM toolbox.

```
# read in lateralized target analysis
erp_c, times = FolderStructure().read_erps(
            erp_name='n2pc_main_task',          # output file name
            cnds=None,                          # no conditions specified
            sjs='all')                          # read in all subjects
```

4.1.5.1 Selecting time windows of interest

A critical step in ERP analysis involves defining appropriate time windows for statistical analysis and component quantification. This selection can be guided by existing literature—particularly advantageous for well-established components like the N2pc—or through data-driven approaches when exploring novel components or populations.

OpenDvM provides flexible functionality for automated window detection based on peak amplitude characteristics. Users can define windows using grand-averaged data pooled across conditions, or obtain condition-specific windows when experimental manipulations are expected to shift component timing. The example in Listing 6 demonstrates window selection using condition-averaged data. This function identifies a 100 ms window centered around the most negative peak in the lateralized difference waveform (contralateral minus ipsilateral activity) within the specified 0-300 ms post-stimulus interval. This data-driven approach ensures that analysis windows are optimally positioned to capture the component of interest while allowing adaptation to specific datasets and experimental designs.

Listing 6 | Selecting component window of interest with select_erp_window in the OpenDvM toolbox.

```
# Select optimal time window for N2pc analysis
 n2pc_win = ERP.select_erp_window(
      erp_c,
      elec_oi=[['PO3','P3'],['PO4','P4']],      # electrode pairs
      method='cnd_avg',                         # average across conditions
      window_oi=(0, 0.3),                       # search window (0-300ms)
      window_size=0.1,                          # 100ms window size
      polarity='neg')                           # negative peak
```

4.1.5.2 Timecourse visualization

To support analysis-specific visualization, OpenDvM includes dedicated plotting functions tailored to the unique characteristics of each analysis type. For ERP analysis, the

`plot_erp_time_course` function provides comprehensive visualization options for lateralized waveforms, enabling researchers to examine both raw contralateral/ipsilateral activity and lateralized difference waveforms.

The examples in Listing 7 demonstrate two complementary visualization approaches using the appropriate electrode pairs for posterior N2pc analysis. The first example displays separate contralateral and ipsilateral waveforms (Figure 2A), while the second shows the lateralized difference waveform with a bounding box highlighting the selected N2pc time window (Figure 2B). This dual visualization approach allows researchers to examine both the underlying hemispheric asymmetries and the resulting lateralized component, providing comprehensive insight into the spatial and temporal characteristics of attentional selection processes.

Listing *7* | ERP visualization with plot_erp_time_course in the OpenDvM toolbox.

```
# Plot contralateral and ipsilateral waveforms separately
plot_erp_time_course(
      erp_c,
      times,
      elec_oi = [['PO3','P3'], ['PO4','P4']], # contra/ipsi electrodes
      lateralized=False,                  # show contra/ipsi separately
      colors=['black', 'grey'])           # contra=black, ipsi=grey

# Plot lateralized difference waveform with time window
plot_erp_time_course(
      erp_c,
      times,
      elec_oi = [['PO3','P3'], ['PO4','P4']], # contra/ipsi electrodes
      lateralized=True,                   # show difference wave
      colors=['blue'],                    # single color for difference
      window_oi=n2pc_win)                 # highlight selected window
```

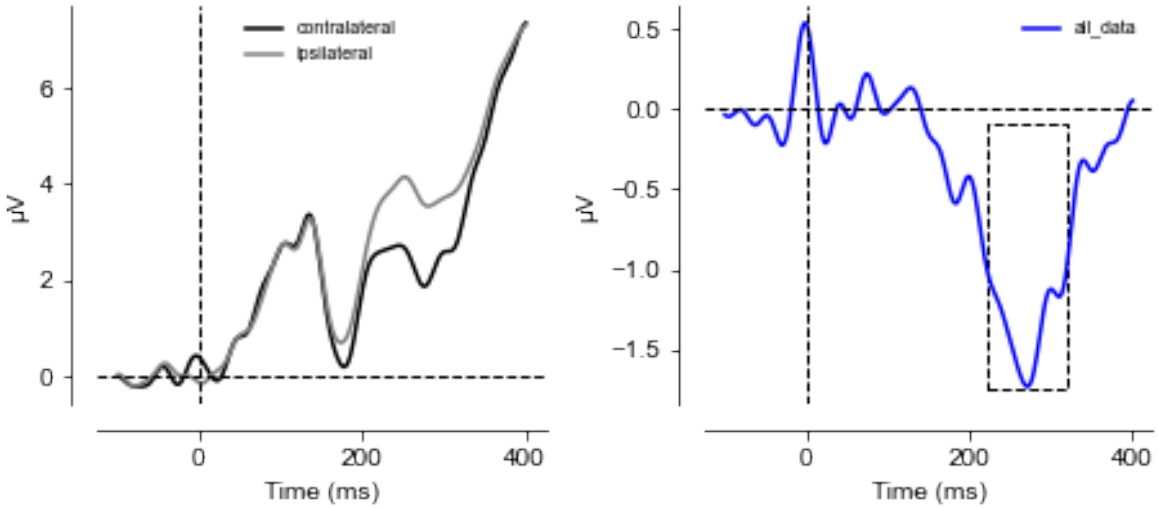


Figure 2 | N2pc waveforms

4.1.5.3 Statistical analysis

In addition to visualization, OpenDvM provides functionality to extract ERP parameters of interest and export them to CSV files for further statistical analysis in external software (e.g., JASP or R). This is particularly useful for workflows that require formal hypothesis testing or integration with broader statistical analysis pipelines.

The `export_erp_metrics_to_csv` function allows users to specify time windows, electrodes of interest, conditions, and desired metrics such as mean amplitude or area under the curve (AUC). The example in Listing 8 demonstrates extracting mean amplitude values for the N2pc component across the selected time window. The resulting CSV file is automatically saved to the `erp/stats/` subfolder within the project directory, facilitating seamless integration with statistical software. The accompanying tutorials demonstrate additional functionality, including jackknife procedures for comparing onset latency differences and other advanced ERP analysis techniques.

Listing 8 | Extraction of erp metrics using export_erp_metrics_to_csv in the OpenDvM toolbox.

```
# Export N2pc mean amplitude to CSV for statistical analysis
ERP.export_erp_metrics_to_csv(
      erp_c,
      window_oi=n2pc_win,
      elec_oi=[['PO3','P3'], ['PO4','P4']],
      cnds=None,                          # all conditions
      method='mean_amp',
      name='n2pc_amp')
```

4.2 TFR Analysis

4.2.1 Time-frequency decomposition and lateralization

Time-frequency representations (TFRs) are a widely used technique in cognitive neuroscience to examine how oscillatory activity evolves over time in response to cognitive events (Cohen, 2014). Unlike ERP analyses, which focus on phase-locked activity, TFRs capture both phase-locked and non-phase-locked changes in neural oscillations, providing a more comprehensive view of neural dynamics (de Vries et al., 2020; Tallon-Baudry & Bertrand, 1999). This makes them particularly valuable for studying attention, where power modulations within specific frequency bands often reveal mechanisms that are not evident in traditional ERP analyses.

For instance, alpha-band desynchronization (8–12 Hz) is commonly associated with attentional engagement and shows systematic lateralization depending on the attended hemifield (Thut et al., 2006; Worden et al., 2000), while theta oscillations (4–8 Hz) have been linked to cognitive control and conflict monitoring processes (Cavanagh & Frank, 2014). TFR analyses thus provide complementary information to ERP components, enabling deeper understanding of the oscillatory mechanisms underlying attentional selection and control.

4.2.2 TFR specific parameters (single subject level)

The `TFR` class accepts several parameters to configure time-frequency decomposition according to analysis requirements. The functionality is heavily inspired by considerations and best practices discussed in Cohen's comprehensive treatment of time-frequency analysis methods (Cohen, 2014). Beyond the required inputs (subject identifier, EEG data as an MNE `Epochs` object, and behavioral data as a pandas `DataFrame`), users can customize frequency analysis settings, baseline correction approaches, and power computation methods.

Key parameters include frequency range specification (`min_freq`, `max_freq`), the number of frequency bins (`num_frex`), and frequency scaling (`'log'` or `'linear'`). Wavelet characteristics are controlled through `cycle_range`, allowing users to balance temporal and frequency resolution according to their analysis goals.

The `base_method` parameter provides flexible baseline correction options: `'trial'` applies trial-specific baselines, `'cnd'` uses condition-specific baselines (each condition corrected by its own baseline), and `'cnd_avg'` applies condition-averaged baselines (each condition corrected by the grand mean baseline, as recommended by Cohen for reducing condition-specific bias). The `power` parameter determines the type of power analysis: `'total'` computes standard time-frequency power, `'induced'` calculates non-phase-locked power after

subtracting the condition-specific evoked response, and `'evoked'` analyzes phase-locked power by averaging trials before decomposition. The example in Listing 10 configures analysis for evoked power across 4-40 Hz using condition-averaged baseline correction, which is particularly useful when standard baseline approaches are compromised by short inter-trial interval

Listing 9 | Initializing a TFR object in the OpenDvM Toolbox.

```
# Initialize TFR class with frequency and baseline settings
tfr_o = TFR(sj, epochs, df,
      min_freq=4,                   # minimum frequency (Hz)
      max_freq=40,                  # maximum frequency (Hz)
      num_frex=25,                  # number of frequency bins
      method = 'wavelet'            # morlet wavelet decomposition
      freq_scaling='log',           # logarithmic frequency spacing
      cycle_range = (3,10)          # wavelet cycles (low to high freq)
      laplacian=False,              # no spatial filtering
      baseline=(-0.4, -0.2),        # baseline: -400 to -200ms
      base_method = 'cnd_avg'       # condition averaged baseline
      power = 'evoked'              # evoked phase-locked power
      report=True)                  # generate quality control report
```

4.2.3 TFR decomposition (single subject level)

Following class initialization, time-frequency decomposition is performed using the `condition_tfrs()` method, which provides flexible options for condition-specific analyses and electrode selection. This function shares many parameters with the ERP analysis pipeline, maintaining consistency across OpenDvM's analytical framework while accommodating the specific requirements of time-frequency analysis.

The function allows users to specify experimental conditions of interest (`cnds`), restrict analyses to particular stimulus positions (`pos_labels`), apply topographic flipping for lateralized analyses (`topo_flip`), and define temporal windows (`window_oi`). The `elec_oi` parameter enables electrode selection, with the trade-off that selecting a subset of electrodes speeds computation but precludes topographical visualization. When `pos_labels` and `topo_flip` arguments are utilized, users can easily limit analysis to topographically aligned stimuli, which is particularly useful for lateralized attention studies.

Listing 10 demonstrates condition-specific analysis comparing target-present versus target-absent trials, while excluding trials with absent distractors to focus on conditions with potential attentional competition:

Listing 10 | Computing condition specific TFRs with condition_erps() in the OpenDvM Toolbox.

```
# Perform time-frequency decomposition across conditions
tfr_o.condition_tfrs(
      pos_labels=None,                    # include all positions
      cnds=dict(target_cnd=['present', 'absent']),    # target conditions
      midline=None,                       # no midline restriction
      topo_flip=None,                     # no topographic flipping
      window_oi=(-0.4, 0.6),              # analysis time window
      elec_oi='all',                      # (enables topo plots)
      excl_factor=dict(dist_cnd=['absent']), # exclude dist-absent trials
      name='target_tfr_evoked')           # output filename
```

4.2.4 Report Generation for Quality Control

As with the ERP module, the TFR class provides automated report generation (enabled via `report=True` during initialization) to support visual inspection and quality control at the single-subject level. For each condition of interest, reports include 2D time-frequency representations collapsed across electrodes, showing how power evolves across time and frequency, as well as individual time-frequency plots for each electrode to inspect spatial patterns and identify potential artifacts.

These reports provide a systematic approach for reviewing preprocessing and analysis outcomes before proceeding to group-level statistical analyses. The HTML reports are automatically saved to the `tfr/` folder, facilitating consistent quality control workflows across datasets.

4.2.5 Visualization and Statistical Analysis (group level)

Group-level TFR analyses follow the same data aggregation approach as ERP analyses. The `read_tfr()` function loads time-frequency representations across subjects, with options to specify participant subsets or particular experimental conditions (Listing 11). Once loaded, the aggregated data are prepared for group-level visualization and statistical analysis, maintaining consistent organization across participants and conditions for downstream processing.

Listing 11 | Reading aggregated TFR data with read_tfrs() in the OpenDvM toolbox.

```
# Load group-level TFR data
tfr_c = FolderStructure().read_tfrs(
      tfr_folder_path=['wavelet'],        # decomposition method folder
      tfr_name='target_tfr_evoked',       # analysis name
      cnds=['present', 'absent'],         # target conditions
      sjs='all')                          # all subjects
```

4.2.5.1 TFR visualization (Add topopplots)

The TFR module provides flexible visualization tools to examine oscillatory dynamics at the group level. A key strength of time-frequency analysis is the ability to visualize data in multiple formats depending on the research question. Users can generate either two-dimensional time-frequency representations showing the full frequency spectrum across time, or one-dimensional time courses where power is averaged across specific frequency bands of interest.

Two complementary visualization approaches demonstrate this flexibility (Listing 12; Figure 3A-B). First, broad-spectrum analysis can reveal unexpected frequency effects or provide comprehensive overviews of oscillatory dynamics (Figure 3A). Second, frequency-specific analysis allows focused examination of theoretically motivated frequency bands (e.g., alpha, theta) and their temporal evolution (Figure 3B).

Listing 12 | TFR visualization with plot_tfr_time_course in the OpenDvM toolbox.

```
# 2D time-frequency representation (broad spectrum view)
plot_tfr_timecourse(tfr_c,
      elec_oi=['PO4','PO3','O1','O2'],          # posterior electrodes
      cnds=['present'],                         # target-present condition
      freq_oi=(4, 40),                          # full frequency range
      lateralized=False,                        # condition-specific
      timecourse='2d',                          # 2D representation
      stats='fdr')                              # FDR-corrected statistic

# 1D time course condition contrast within alpha band (frequency-specific)
plot_tfr_timecourse(tfr_c,
      elec_oi=['O1','PO3','O2','PO4'],          # posterior electrodes
      cnds=['present','absent'],                # condition contrast
      freq_oi=(8, 12),                          # alpha frequency range
      lateralized=False,                        # condition comparison
      timecourse='1d',                          # 1D time course
      stats='fdr')                              # FDR-corrected statistic
```

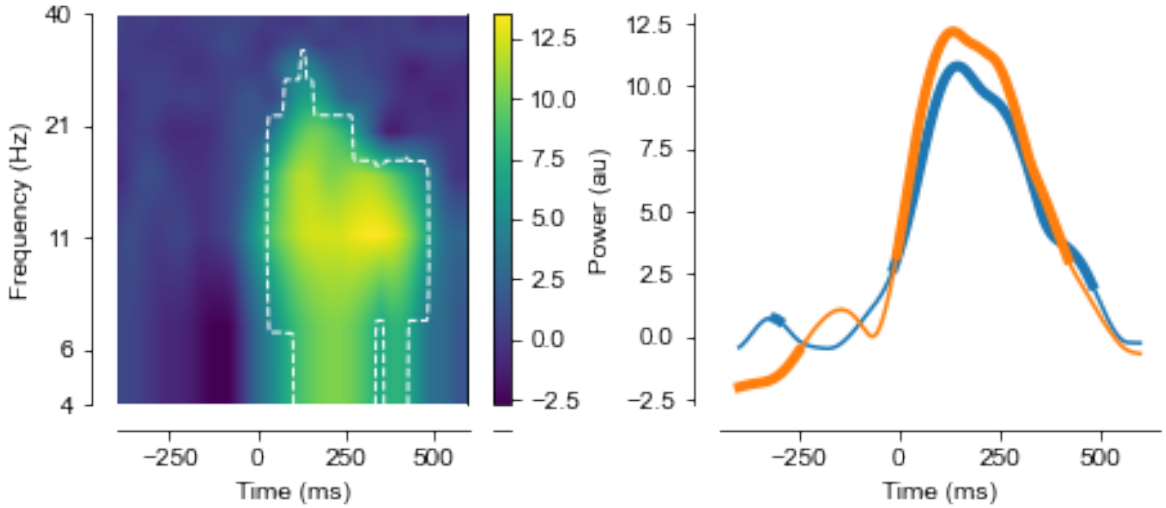


Figure 3: TFR results

4.2.5.2 Statistical analysis

Time-frequency data present unique statistical challenges due to the multiple comparisons problem across both time and frequency dimensions. While OpenDvM provides functionality to export mean power values for external statistical software (similar to ERP analyses), the toolbox emphasizes permutation-based statistical approaches that control for multiple comparisons across the time-frequency space.

The statistical testing options integrate directly with visualization functions, allowing users to overlay significant time points or time-frequency clusters onto their plots. For two-dimensional representations, significant clusters are highlighted across the time-frequency plane, while one-dimensional plots show significant time periods within the selected frequency band. This approach provides both statistical rigor and intuitive visualization of results.

4.3 Backward decoding analyses

4.3.1 Multivariate decoding approach

Backward decoding analysis (also called multivariate pattern analysis or MVPA) aims to determine whether neural activity can reliably discriminate between experimental conditions, such as stimulus categories or spatial positions. This approach is termed 'backward' because it decodes experimental variables from neural data, in contrast to 'forward' encoding models that predict neural responses from stimulus features. OpenDvM implements these machine learning approaches using scikit-learn (Pedregosa et al., 2011), a well-established Python package, ensuring robust and validated algorithms.

In this framework, a classifier is trained to learn patterns in EEG data that distinguish between conditions, with performance evaluated on independent data. Compared to traditional approaches such as ERPs, which are often limited to lateralized designs (e.g., left vs. right targets), decoding methods can capture neural signatures across richer spatial or feature spaces. This enables analysis of circular stimulus arrays or simultaneous discrimination of multiple stimulus features—paradigms that are increasingly common in attention research.

To ensure valid and interpretable results, OpenDvM enforces balanced experimental designs by equalizing trial numbers across conditions through within-class undersampling, where surplus trials are randomly discarded. While this prevents classifier bias from unequal trial numbers, substantial trial loss may occur with heavily unbalanced designs. OpenDvM supports both within-dataset analysis using k-fold cross-validation and cross-generalization across independent datasets (e.g., training on localizer data and testing on search tasks), enabling investigation of both condition-specific performance and cross-context generalizability of neural representations.

4.3.2 BDM specific parameters (single subject level)

Backward decoding analyses require specification of several parameters that critically shape classification performance. Users must indicate which experimental variable to decode by specifying the corresponding behavioral file column (`to_decode`), with additional options available for tailoring the analysis to specific datasets and research questions.

The choice of classifier (`classifier`) and performance metric (`metric`) are specified during initialization. By default, OpenDvM uses linear discriminant analysis (LDA) and area under the curve (AUC) as the performance metric. LDA has been shown to achieve robust classification performance on EEG/MEG data while offering computational advantages over more complex classifiers like support vector machines (Grootswagers et al., 2017), though SVM or Gaussian Naive Bayes classifiers can alternatively be selected.

The `data_type` parameter determines whether decoding operates on raw voltage values (`'raw'`) for broadband analysis or on time-frequency representations (`'tfr'`) to focus on specific oscillatory signatures (see section 4.3.5). Cross-validation is configured via `nr_folds`, which determines the proportion of data used for testing in each iteration (e.g., 10 folds means 10% serves as test data iteratively, while 2 folds uses 50% for testing). Additional parameters include the number of cross-validation repetitions (`avg_runs`) for stability, trial averaging (`avg_trials`) to boost signal-to-noise ratios by averaging across trials prior to other

preprocessing steps, electrode selection (`elec_oi`), baseline correction windows, and optional dimensionality reduction via PCA. The example in Listing 13 initializes a decoding analysis to classify distractor images using 10-fold cross-validation with a -200 to 0 ms baseline window:

Listing 13 | Initializing a BDM object in the OpenDvM Toolbox.

```
# Initialize BDM class for distractor image classification
bdm_o = BDM(sj, epochs, df,
      to_decode = 'dist_img',             # decode distractor image type
      baseline=(-0.2, 0),                 # baseline correction window
      nr_folds=10,                        # 10% test data per fold
      elec_oi='all',                      # use all electrodes
      avg_runs=1,                         # single run
      avg_trials=1,                       # no trial averaging
      data_type='raw',                    # raw voltage (broadband)
      classifier='lda',                   # linear discriminant analysis
      metric='auc')                       # classification metric
```

4.3.3 Decoding within conditions (single subject level)

The `classify()` method performs decoding analyses within specified experimental conditions, enabling comparison of neural representations across different task contexts or examination of decoding performance within individual experimental phases. To ensure valid comparisons across conditions, the toolbox provides trial exclusion options via `excl_factor`. The example in Listing 14 demonstrates decoding distractor images within both localizer and main task sessions, while excluding target-present trials from the main task to match the trial structure of the localizer session:

Listing 14 | Condition specific decoding with classify() in the OpenDvM Toolbox.

```
# Decode distractor images within experimental conditions
bdm_o.classify(
      cnds=dict(block_type=['loc', 'resp']),    # localizer & main task
      window_oi=(-0.2, 0.5),                    # analysis time window
      labels_oi='all',                          # decode all image types
      bdm_name='img_dec',                       # analysis name
      excl_factor=dict(target_cnd=['present'])) # exclude target trials
```

The toolbox automatically balances trial numbers across conditions and classes before classifier training. Additional functionality includes spatial analysis via `split_fact` (e.g., `split_fact=dict(dist_loc=[0,2,4,6])`) to perform location-specific decoding that is then averaged across spatial positions. The method also supports advanced analyses such as

generalization across time (GAT; see section 4.3.6) and decoding with progressively reduced trial subsets to assess minimal trial requirements for reliable classification.

4.3.4 Cross-condition decoding (single subject level)

Cross-condition decoding trains the classifier on one experimental condition and tests it on another, allowing assessment of how well neural representations generalize across different task contexts. For example, a classifier can be trained on localizer data and tested on main experiment data to examine whether stimulus representations learned in one context transfer to another.

In OpenDvM, cross-condition decoding is implemented by passing conditions as nested lists to the `cnds` argument. The first nested list specifies the training condition(s), while the second specifies the testing condition(s), establishing completely independent training and testing datasets (see Listing 15). This nested list syntax ensures that trials from the 'loc' condition are used exclusively for training the classifier, while trials from the 'resp' condition serve only as independent test data. The same principles of balanced trial counts and optional trial exclusion apply, ensuring that cross-condition comparisons are not confounded by imbalanced experimental designs. Results are automatically saved to the `bdm/` folder for subsequent group-level analysis.

Listing 15 | Cross-condition decoding with classify() in the OpenDvM Toolbox.

```
# Cross-condition decoding: train on localizer, test on main task
bdm_o.classify(
      cnds=dict(block_type=[['loc'], ['resp']]),# train 'loc', test 'resp'
      window_oi=(-0.2, 0.5),                    # analysis time window 1
      labels_oi='all',                          # decode all image types
      bdm_name='cross_dec_stim',                # analysis name
      excl_factor=dict(target_cnd=['present'])) # exclude target trials
```

4.3.5 Time-frequency based decoding

While OpenDvM performs decoding by default on raw EEG time series (broadband activity), it also supports classification in the time-frequency domain. This enables investigation of whether specific oscillatory frequency bands drive classification performance, potentially revealing frequency-specific coding of experimental variables. OpenDvM provides two approaches for time-frequency decoding. First, users can independently create a TFR object (see section 4.2), then pass it to the BDM class via the `tfr` attribute. Alternatively, time-

frequency parameters can be specified directly during BDM initialization, allowing the toolbox to perform decomposition internally before classification.

The example in Listing 16 demonstrates the integrated approach, where TFR parameters are passed directly to the BDM class. The data are decomposed into 8 linearly-spaced frequency bins between 4-40 Hz, and the resulting time-frequency representation serves as input for decoding. This approach extends the flexibility of multivariate analyses in OpenDvM by making it possible to track not only *where* and *when* decoding information arises, but also *which frequency bands* contribute most strongly to classification performance.

Listing 16 | Condition specific TFR based decoding with classify() in the OpenDvM Toolbox.

```
# Initialize BDM with integrated TFR decomposition
bdm_o = BDM(sj, epochs, df,
      to_decode = 'dist_img',
      baseline=(-0.3, -0.1), # baseline window
      data_type='tfr', # time-frequency decoding
      min_freq=4, # minimum frequency
      max_freq=40, # maximum frequency
      num_frex=8, # number of frequency bins
      freq_scaling='linear', # linear frequency spacing
      cycle_range=(3, 10)) # wavelet cycles

# Classify using time-frequency representations
bdm_o.classify(
      cnds=dict(block_type=['loc', 'resp']),
      window_oi=(-0.2, 0.5),
      bdm_name='tfr_img_dec',
      excl_factor= dict(target_cnd=['present']))
```

4.3.6 Temporal Generalization

Beyond conventional time-resolved decoding, OpenDvM supports generalization across time (GAT), also known as temporal generalization analysis. In this approach, the classifier is trained at one time point and tested across all other time points in the trial, producing a two-dimensional train × test matrix that reveals how stable or dynamic neural representations are over time (King & Dehaene, 2014).

Whereas standard decoding focuses on when information can be discriminated, temporal generalization reveals how long neural codes remain stable and whether they evolve into new representational formats. Strong diagonal patterns in the GAT matrix indicate transient, time-specific representations, while broader off-diagonal generalization suggests

sustained or reactivated codes that extend beyond the training interval (King & Dehaene, 2014; Stokes et al., 2013).

Temporal generalization is computationally demanding as it requires training and testing at every possible time-point combination. OpenDvM provides flexible options within the `classify()` method: setting `GAT=True` performs full temporal generalization across the entire analysis window, while specifying custom training and testing windows as a tuple (e.g., `GAT=((0.1, 0.3),(-0.2, 0.5))`) enables more targeted and computationally efficient analyses. Users can also reduce computational demands through downsampling at class initialization.

4.3.7 Topographic maps

Add

4.3.8 Report Generation for Quality Control

What to add here?

4.3.9 Visualization and Statistical Analysis (group level)

Group-level decoding analyses begin by aggregating single-subject results using the `read_bdm()` function, which loads decoding output in a format suitable for visualization and statistical testing. By default, a single analysis is loaded for plotting or statistical comparison.

Unlike the ERP and TFR modules, the BDM module also supports simultaneous import of multiple analyses, provided that time windows and subject sets overlap. This enables direct comparison of different analysis pipelines within the same visualization or statistical test—for example, contrasting decoding performance with and without trial averaging, or comparing different classifier types (see Listing 17).

Listing 17 | Reading aggregated BDM data with read_bdm() in the OpenDvM toolbox.

```
# Load single decoding analysis
bdm_c = FolderStructure().read_bdm(
      bdm_folder_path=['dist_img', 'all_elecs'],# analysis subfolder path
      bdm_name='within_loc',                    # single analysis
      sjs='all')                                # all subjects

# Load multiple analyses for comparison
bdm_c = FolderStructure().read_bdm(
      bdm_folder_path=['dist_img', 'all_elecs'], # analysis subfolder path
      bdm_name=['within_loc', 'within_loc_avg4'],# compare two analyse
      sjs='all')                                  # all subjects
```

4.3.9.1 BDM Visualization

OpenDvM provides specialized plotting functions for multivariate decoding that accommodate different analysis types. The `plot_bdm_time_course()` function supports three visualization modes: one-dimensional time courses showing classification performance across time (Figure 4A), two-dimensional temporal generalization matrices (`timecourse='2d_GAT';` see Figure 4B), and time-frequency decoding heatmaps (`timecourse='2d_tfr';` see Figure 4C).

For time course visualizations, multiple analyses can be directly compared within the same figure, enabling assessment of how different preprocessing choices or analysis parameters affect decoding performance. For two-dimensional representations, specific conditions are selected via the `cnds` argument, and plotting is restricted to a single condition at a time (see Listing 18).

Listing 18 | | BDM visualization with plot_bdm_time_course in the OpenDvM toolbox.

```
# 1D time course comparing multiple analyses (using previously loaded data)
plot_bdm_time_course(bdm,
      colors=['green', 'red'],
      show_SE=False,
      smooth=False,
      stats='perm',                             # cluster-based permutation
      timecourse='1d')                          # time course plot

# 2D time-frequency decoding (requires loading TFR-based decoding results)
plot_bdm_time_course(bdm_tfr,
      timecourse='2d_tfr',                      # frequency decomposition
      stats='perm',                             # cluster-based permutation
      cnds=['resp'])                            # select specific condition

# 2D temporal generalization matrix (requires loading GAT results)
plot_bdm_time_course(bdm_gat,
      timecourse='2d_GAT',                      # GAT visualization
      stats='fdr',                              # false discovery rate
      cnds=['loc'])                             # select specific condition
```

4.3.9.2 Statistical analysis

Multivariate decoding data present similar multiple comparisons challenges as time-frequency analyses, particularly when evaluating significance across many time points or in two-dimensional spaces (GAT matrices, time-frequency decoding). OpenDvM emphasizes

permutation-based statistical approaches that control for multiple comparisons while maintaining sensitivity to meaningful effects.

Statistical testing integrates directly with visualization functions, allowing users to overlay significance results onto plots. Cluster-based permutation testing can be applied to both one-dimensional time courses and two-dimensional representations, identifying temporally (or spatiotemporally) contiguous regions of significant decoding. Alternative correction methods include FDR correction and paired t-tests for more targeted comparisons. Significant clusters or time points are automatically highlighted in the corresponding visualization, providing both statistical rigor and intuitive interpretation of results. For analyses requiring external statistical software, decoding performance metrics can be exported using `export_bdm_to_csv()`, similar to the ERP and TFR modules

4.4 Spatial Channel Tuning Functions (CTF)

4.4.1 Forward encoding modelling

Multivariate analyses can be broadly divided into backward decoding models (BDMs) and forward encoding models (FEMs), each providing complementary insights into the representational content of EEG signals. As described in the previous section, BDMs attempt to discriminate between a fixed set of experimental conditions based on multivariate EEG activity patterns. Successful decoding demonstrates that EEG carries information about those conditions, with the temporal profile of decoding accuracy indicating when such information becomes available.

FEMs reverse this inference (Brouwer & Heeger, 2009). Instead of predicting condition labels from neural activity, FEMs describe the continuous relationship between multivariate activity and a hypothesized representational dimension. One widely used implementation is the channel tuning function (CTF) approach, which in spatial attention research models the circular arrangement of stimulus locations. By fitting a forward model, researchers can reconstruct CTFs that reveal graded, continuous spatial selectivity—for example, alpha-band responses peaking contralateral to the attended location and falling off smoothly across other positions (Foster et al., 2016, 2017). This distinction makes FEMs a powerful complement to decoding analyses. Whereas decoding establishes whether information is present in EEG signals, FEMs characterize how that information is represented, allowing reconstruction of the representational geometry underlying attentional selection. Building on seminal work by Foster and colleagues (Foster et al., 2016, 2017), OpenDvM implements a streamlined pipeline for

CTF analyses—to the best of our knowledge, the first fully integrated, open-source Python pipeline —enabling researchers to reconstruct spatial tuning functions from raw EEG or time-frequency decomposed data without relying on MATLAB-based toolboxes (Fahrenfort et al., 2018).

4.4.2 CTF specific parameters (single subject level).

The CTF class builds on the logic of forward encoding models by reconstructing spatial channel responses from the distribution of EEG activity. Beyond the standard parameters (subject identifier, MNE `Epochs` object, behavioral `DataFrame`, and the variable to decode), several parameters specific to forward encoding must be specified at initialization.

Central to CTF analyses is the specification of `nr_bins` and `nr_chans`, which define the number of discrete angular locations in the stimulus space and the number of modeled spatial channels, respectively. These parameters should match the experimental design: for stimuli arranged in a circular array at eight possible locations, both values would typically be set to eight. Each spatial channel's tuning function is generated using a sinusoidal basis function, with selectivity controlled by the `sin_power` parameter. Following Foster et al. (2016), each channel's response profile is modeled as a half-sinusoid raised to a power (commonly 7), producing sharply tuned functions that are circularly shifted to cover the entire stimulus space. Alternatively, users can set `delta=True` to employ Kronecker delta (stick) basis functions instead. Unlike graded sinusoidal functions, stick functions specify a response of 1 for the channel's preferred location and 0 elsewhere. This allows researchers to test whether graded CTF profiles reflect true graded selectivity in the neural data or are imposed by the graded basis function itself (Foster et al., 2017). If spatially selective activity is genuinely graded, CTFs reconstructed with stick functions will still show smooth tuning profiles; if not, only the channel tuned to the attended location will show a response.

For analyses involving time-frequency decomposition, TFR parameters (`min_freq`, `max_freq`, `num_frex`, `freq_scaling`) are specified during initialization. However, whether encoding operates on raw voltage, specific frequency bands, or across multiple frequencies is determined at the analysis stage (see section 4.4.3), providing flexibility to explore different representational spaces from the same initialized object.

Additional parameters include dimensionality reduction via `pca_cmp`, electrode selection via `elec_oi`, and baseline correction through the `baseline` parameter. The `avg_ch` flag controls output format: when `True`, all reconstructed channels are averaged into a single

tuning function per condition; when `False`, individual channel responses are retained. The example in Listing 19 initializes a CTF object with parameters for eight spatial positions, including TFR settings that enable subsequent frequency-specific analyses.

Listing 19 | Initializing a CTF object in the OpenDvM Toolbox.

```
# Initialize CTF class for spatial tuning reconstruction
ctf_o = CTF(sj, epochs, df,
      to_decode='img_loc',                      # decode spatial location
      nr_bins=8,                                # 8 stimulus positions
      nr_chans=8,                               # 8 spatial channels
      elec_oi='all',                            # use all electrodes
      freq_scaling='linear',                    # linear frequency spacing
      min_freq=4,                               # minimum frequency
      max_freq=30,                              # maximum frequency
      num_frex=12,                              # 12 frequency bins
      sin_power=7,                              # channel tuning sharpness
      delta=False,                              # use graded basis functions
      avg_ch=True)                              # average to single CTF
```

4.4.3 Within condition CTFs (single subject level).

The `spatial_ctf()` method reconstructs channel tuning functions within specified experimental conditions using cross-validation. To ensure balanced estimates, OpenDvM automatically matches trial numbers across conditions and position bins. By default, data are split into three blocks (via the `class` parameter), with two blocks serving as training data and one as test data. This procedure is rotated until each block has served as the test set, with the entire process repeated multiple times as specified during initialization.

CTF reconstruction can be applied to different signal representations via the `freqs` parameter. Setting `freqs='raw'` operates on broadband EEG voltage, while `freqs='main_param'` scans across the full range of frequencies specified during class initialization (see Listing 20). Alternatively, specific frequency bands can be targeted by passing a dictionary (e.g., `freqs=dict(alpha=[8,12])`), enabling focused examination of oscillatory contributions to spatial tuning. This flexibility allows researchers to systematically explore which signal representations carry the strongest spatial selectivity.

When frequency-based reconstruction is performed (`freqs='main_param'` or specific frequency bands), OpenDvM automatically computes both total and evoked power representations, providing complementary perspectives on spatial selectivity. Total power is computed by squaring the complex magnitude of the analytic signal and then averaging across

trials, capturing ongoing oscillatory activity irrespective of phase consistency. Evoked power is calculated by first averaging the complex analytic signal across trials before squaring, isolating only phase-locked activity with consistent phase relationships to stimulus onset (Foster et al., 2016, 2017). Critically, these power types reveal different aspects of spatial processing: both evoked and total power across low frequencies (<10 Hz) have been shown to transiently track stimulus location following onset, whereas only total alpha-band power (8-12 Hz) tracks sustained covert attention and the content of working memory (Foster et al., 2016, 2017; van Moorselaar et al., 2018).

Analyses can include all position bins (`pos_labels='all'`) or be restricted to specific spatial locations, with optional trial exclusion via `excl_factor` to match trial structures across conditions.

Listing 20 | Condition specific encoding with spatial_ctf() across a range of frequencies in the OpenDvM Toolbox.

```
# Reconstruct CTFs across frequency range
ctf_o.spatial_ctf(
      pos_labels='all',                          # all positions
      cnds=dict(block_type=['loc']),             # localizer condition
      window_oi=(-0.2, 0.4),                     # time window
      freqs='main_param',                        # scan all frequencies
      downsample=4,                              # temporal downsampling
      name='img_ctf',                            # output name
      excl_factor=dict(target_cnd=['present'])) # exclude target trials
```

4.4.4 Cross-condition encoding (single subject level)

A common analytical strategy is to first establish reliable spatial tuning within an independent localizer condition before testing generalization to experimental conditions of interest. For raw voltage-based CTF reconstruction, applying a low-pass filter (typically 6-8 Hz) often yields optimal results by isolating slow spatial selectivity signals while reducing high-frequency noise. Dimensionality reduction via PCA can further improve signal-to-noise ratios, though the optimal number of components may vary across datasets.

As with all OpenDvM classes, parameters can be dynamically updated (see section 3.4) before analysis by modifying the CTF object's attributes. The example in Listing 21 demonstrates this workflow, using 16 PCA components and a 7 Hz low-pass filter: first reconstructing CTFs within the localizer condition to verify reliable spatial tuning, then testing whether these spatial representations generalize to the main task using cross-condition reconstruction.

The nested list syntax in the cross-condition analysis mirrors the BDM approach (section 4.3.4), where the first nested list specifies training data and the second specifies testing data. This enables assessment of whether spatial representations learned in one context transfer to another, revealing the generalizability of neural tuning patterns across task demands.

Note: While CTF supports temporal generalization analysis via `gat=True` (similar to BDM), this functionality is computationally intensive and rarely necessary, as standard time-resolved reconstruction captures the temporal evolution of spatial tuning effectively.

Listing 21 | Cross-condition encoding with spatial_ctf() in the OpenDvM Toolbox.

```
# Configure preprocessing for raw voltage CTF analysis
ctf_o.pca_cmp = 16 # reduce to 16 PCA components
ctf_o.filter = 7 # 7 Hz low-pass filter
ctf_o.baseline = (-0.2, 0) # baseline correction window

# Reconstruct CTFs within localizer condition
ctf_o.spatial_ctf(
      pos_labels='all',
      cnds=dict(block_type=['loc']), # within localizer
      window_oi=(-0.2, 0.4),
      freqs='raw', # broadband voltage
      downsample=4, # temporal downsampling
      name='loc_ctf',
      excl_factor=dict(target_cnd=[‘present’]))

# Cross-condition reconstruction: train on localizer, test on main task
ctf_o.spatial_ctf(
      pos_labels='all',
      cnds=dict(block_type=[['loc'], ['resp']]), # train 'loc', test 'resp'
      window_oi=(-0.2, 0.4),
      freqs='raw',
      downsample=4,
      name='target_ctf',
      excl_factor=dict(target_cnd=['present']))
```

4.4.5 Report Generation for Quality Control

As with other OpenDvM modules, the CTF class provides automated report generation (enabled via `report=True` during initialization) to support visual inspection of reconstruction quality at the single-subject level. CTF reports include distinct visualizations for each outcome measure (raw EEG, total power, and evoked power when applicable).

For each measure, reports display the slope of the channel tuning function over time, revealing when spatial selectivity emerges and how it evolves. When multiple frequencies are analyzed, slopes are presented as heatmaps across time and frequency. Additionally, heatmaps show the full channel tuning function profile across time for each condition, allowing inspection of both the timing and spatial specificity of reconstructed selectivity.

These visualizations enable users to verify that spatial tuning onsets at theoretically expected time points and exhibits appropriate magnitude. Reliable spatial tuning typically produces slopes in the range of 0.05 to 0.2, with clear peaks corresponding to attended or encoded locations. Reports are automatically saved as HTML files in the `ctf/` folder, facilitating systematic quality assessment before group-level analyses. ADD TOPOLOTS AND Individual channel plots.

4.4.6 Visualization and Statistical Analysis (group level)

Group-level CTF analyses begin by loading single-subject results using the `read_ctfs()` function (Listing 22). Critically, users can choose between two output types via the `output_type` parameter: loading the full reconstructed channel tuning functions themselves (`output_type='ctf'`) for visualization, or loading summary parameters that describe the tuning function properties (`output_type='param'`) for statistical analysis. This approach enables both visualization of spatial tuning profiles and statistical testing of specific tuning properties, as described in the following sections.

By default, the parameter extraction returns the slope of the tuning curve, which quantifies the strength of spatial selectivity. Alternative parameters—such as the amplitude or width of the tuning curve—can be specified during class initialization, enabling different characterizations of spatial tuning depending on the research question.

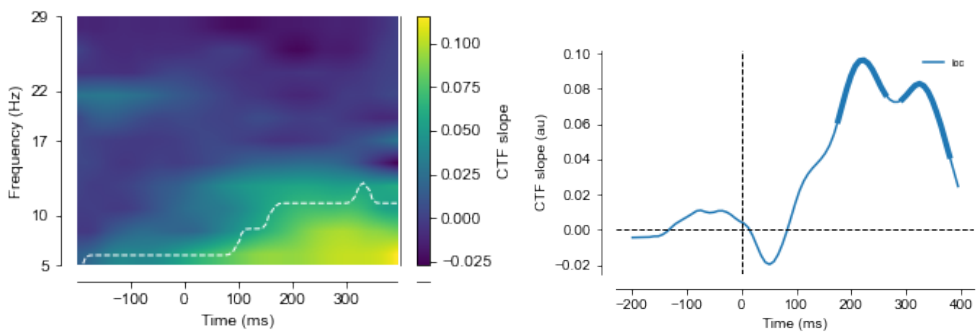
22 | Reading aggregated CTF data with read_ctf() in the OpenDvM toolbox.

```
# Load reconstructed CTFs (set output_type='ctf' for full profiles,
# 'param' for summary parameters like slope)
ctfs = FolderStructure().read_ctfs(
      ctf_folder_path=['img_loc'],                    # analysis subfolder
      output_type='ctf',                              # 'ctf' or 'param'
      ctf_name='img_ctf',                             # analysis name
      sjs='all')                                      # all subjects
```

4.4.6.1 CTF visualization

OpenDvM provides flexible visualization options for channel tuning functions through the `plot_ctf_time_course()` function, which supports three complementary display modes depending on the analysis type and research question. Visualization mode is determined by the `timecourse` parameter in combination with the loaded data type (`output_type='ctf'` or `'param'`).

For analyses of tuning function parameters (slope, amplitude, width), one-dimensional time courses display parameter evolution across the trial (`timecourse='1d'`), while two-dimensional heatmaps show parameter values across time and frequency (`timecourse='2d_tfr'`). These visualizations require loading parameter data (`output_type='param'`). For examining the full spatial tuning profile, two-dimensional CTF heatmaps (`timecourse='2d_ctf'`) display the reconstructed channel response function across time, revealing both the magnitude and spatial specificity of tuning. This visualization requires loading full CTF profiles (`output_type='ctf'`). These visualization modes enable comprehensive examination of spatial selectivity from both quantitative (parameter-based) and qualitative (profile-based) perspectives, with integrated statistical testing for parameter-based analyses (Figure X).



4.4.6.2 Statistical analysis

Statistical inference for CTF parameters integrates directly with visualization functions, following the same approach as time-frequency and decoding analyses. When analyzing tuning function parameters (slope, amplitude, width), multiple correction methods are available: cluster-based permutation testing (`stats='perm'`) for temporally contiguous effects in one-dimensional time courses, false discovery rate correction (`stats='fdr'`), and paired t-tests (`stats='ttest'`) for more targeted comparisons.

The choice of statistical method depends on the visualization type and hypothesis. Cluster-based permutation testing is particularly appropriate for one-dimensional parameter time courses where temporal contiguity is meaningful, while FDR or t-test correction may be more suitable for two-dimensional time-frequency representations. Significant clusters or time points are automatically highlighted in the corresponding visualization, providing integrated statistical inference and intuitive result interpretation.

For analyses requiring external statistical software, CTF parameters can be exported using `export_ctf_to_csv()`, similar to the ERP, TFR, and BDM modules. Note that statistical testing is only applicable to parameter-based analyses; full CTF profile visualizations (`timecourse='2d_ctf'`) serve primarily for qualitative inspection of spatial tuning patterns.

Discussion

OpenDvM provides a unified framework for applying diverse analytical approaches to EEG data from attention research. The toolbox's core contribution is not methodological innovation but rather practical integration: by enforcing consistent syntax, data handling, and preprocessing conventions across event-related potentials, time-frequency analysis, multivariate decoding, and forward encoding models, OpenDvM enables researchers to seamlessly combine multiple analytical perspectives on the same dataset. This unified interface reduces the cognitive and technical burden of switching between different toolboxes, each with distinct parameter conventions and output formats, facilitating more integrative analyses within attention research.

OpenDvM complements rather than replaces established frameworks like MNE-Python and scikit-learn. The toolbox leverages these robust, well-maintained packages for core functionality while providing a specialized layer optimized for attention research workflows. While developed and validated in attention research contexts, OpenDvM's architecture is sufficiently general to accommodate diverse experimental paradigms and research questions. The modular design of analysis classes and the flexibility of parameter specification enable straightforward adaptation to other domains requiring similar analytical pipelines.

Accompanying tutorials extending beyond this methods paper provide detailed guidance on advanced functionality not demonstrated here, including jackknife procedures for comparing onset latencies, channel tuning reconstructions for individual stimulus locations, and integration strategies for complex multi-method analyses. These supplementary resources support users in adapting OpenDvM to specific research contexts and questions.

References

Alilović, J., van Moorselaar, D., Graetz, M., van Gaal, S., & Slagter, H. A. (2021). Representational dynamics preceding conscious access. *Neuroimage*, *230*, 117789.

Brouwer, G. J., & Heeger, D. J. (2009). Decoding and reconstructing color from responses in human visual cortex. *Journal of Neuroscience*, *29*(44), 13992-14003.

Cavanagh, J. F., & Frank, M. J. (2014). Frontal theta as a mechanism for cognitive control. *Trends in cognitive sciences*, *18*(8), 414-421.

Cohen, M. X. (2014). *Analyzing neural time series data: theory and practice*. MIT press.

de Vries, I. E., Slagter, H. A., & Olivers, C. N. (2020). Oscillatory control over representational states in working memory. *Trends in cognitive sciences*, *24*(2), 150-162.

Delorme, A., & Makeig, S. (2004). EEGLAB: an open source toolbox for analysis of single-trial EEG dynamics including independent component analysis. *Journal of neuroscience methods*, *134*(1), 9-21.

Duncan, D., Theeuwes, J., & van Moorselaar, D. (2023). The electrophysiological markers of statistically learned attentional enhancement: evidence for a saliency-based mechanism. *Journal of cognitive neuroscience*, *35*(12), 2110-2125.

Duncan, D., van Moorselaar, D., & Theeuwes, J. (2023). Pinging the brain to reveal the hidden attentional priority map using encephalography. *Nature Communications*, *14*(1), 4749.

Eimer, M. (1996). The N2pc component as an indicator of attentional selectivity. *Electroencephalography and clinical neurophysiology*, *99*(3), 225-234.

Fahrenfort, J. J., Grubert, A., Olivers, C. N., & Eimer, M. (2017). Multivariate EEG analyses support high-resolution tracking of feature-based attentional selection. *Scientific reports*, *7*(1), 1886.

Fahrenfort, J. J., Van Driel, J., Van Gaal, S., & Olivers, C. N. (2018). From ERPs to MVPA using the Amsterdam decoding and modeling toolbox (ADAM). *Frontiers in neuroscience*, *12*, 368.

Foster, J. J., Sutterer, D. W., Serences, J. T., Vogel, E. K., & Awh, E. (2016). The topography of alpha-band activity tracks the content of spatial working memory. *Journal of Neurophysiology*, *115*(1), 168-177.

Foster, J. J., Sutterer, D. W., Serences, J. T., Vogel, E. K., & Awh, E. (2017). Alpha-band oscillations enable spatially and temporally resolved tracking of covert spatial attention. *Psychological Science*, *28*(7), 929-941.

Gramfort, A., Luessi, M., Larson, E., Engemann, D. A., Strohmeier, D., Brodbeck, C., Goj, R., Jas, M., Brooks, T., & Parkkonen, L. (2013). MEG and EEG data analysis with MNE-Python. *Frontiers in neuroinformatics*, *7*, 267.

Grootswagers, T., Wardle, S. G., & Carlson, T. A. (2017). Decoding dynamic brain patterns from evoked responses: a tutorial on multivariate pattern analysis applied to time series neuroimaging data. *Journal of cognitive neuroscience*, *29*(4), 677-697.

Hickey, C., Di Lollo, V., & McDonald, J. J. (2009). Electrophysiological indices of target and distractor processing in visual search. *Journal of cognitive neuroscience*, *21*(4), 760-775.

Hillyard, S. A., & Anllo-Vento, L. (1998). Event-related brain potentials in the study of visual selective attention. *Proceedings of the National Academy of Sciences*, *95*(3), 781-787.
Huang, C., van Moorselaar, D., Foster, J., Donk, M., & Theeuwes, J. (2025). Neural mechanisms of learned suppression uncovered by probing the hidden attentional priority map. *Elife*, *13*, RP98304.
King, J.-R., & Dehaene, S. (2014). Characterizing the dynamics of mental representations: the temporal generalization method. *Trends in cognitive sciences*, *18*(4), 203-210.
Li, A.-S., Theeuwes, J., & van Moorselaar, D. (2024). Neural representations of predicted events: Evidence from time-resolved EEG decoding. *Elife*, *13*.
Lien, M.-C., Ruthruff, E., Goodin, Z., & Remington, R. W. (2008). Contingent attentional capture by top-down control settings: converging evidence from event-related potentials. *Journal of Experimental Psychology: Human Perception and Performance*, *34*(3), 509.
Luck, S. J. (2012). Electrophysiological correlates of the focusing of attention within complex visual scenes: N2pc and related ERP components. *The Oxford handbook of event-related potential components*, 329-360.
Luck, S. J. (2014). *An introduction to the event-related potential technique*. MIT press.
Luck, S. J., & Hillyard, S. A. (1994a). Electrophysiological correlates of feature analysis during visual search. *Psychophysiology*, *31*(3), 291-308.
Luck, S. J., & Hillyard, S. A. (1994b). Spatial filtering during visual search: evidence from human electrophysiology. *Journal of Experimental Psychology: Human Perception and Performance*, *20*(5), 1000.
Mathôt, S., Schreij, D., & Theeuwes, J. (2012). OpenSesame: An open-source, graphical experiment builder for the social sciences. *Behavior research methods*, *44*(2), 314-324.
Mazza, V., Turatto, M., Umiltà, C., & Eimer, M. (2007). Attentional selection and identification of visual objects are reflected by distinct electrophysiological responses. *Experimental Brain Research*, *181*(3), 531-536.
Oostenveld, R., Fries, P., Maris, E., & Schoffelen, J.-M. (2011). FieldTrip: open source software for advanced analysis of MEG, EEG, and invasive electrophysiological data. *Computational intelligence and neuroscience*, *2011*, 1.
Oxner, M., van Moorselaar, D., Müller, M. M., & Theeuwes, J. (2025). A flash in the pan? Distractor suppression cannot be inferred from the early lateralized positivity. *Journal of cognitive neuroscience*, 1-19.
Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., & Dubourg, V. (2011). Scikit-learn: Machine learning in Python. *Journal of machine learning research*, *12*(Oct), 2825-2830.
Stokes, M. G., Kusunoki, M., Sigala, N., Nili, H., Gaffan, D., & Duncan, J. (2013). Dynamic coding for cognitive control in prefrontal cortex. *Neuron*, *78*(2), 364-375.
Tallon-Baudry, C., & Bertrand, O. (1999). Oscillatory gamma activity in humans and its role in object representation. *Trends in cognitive sciences*, *3*(4), 151-162.
Thut, G., Nietzel, A., Brandt, S. A., & Pascual-Leone, A. (2006). α-Band electroencephalographic activity over occipital cortex indexes visuospatial attention bias and predicts visual target detection. *Journal of Neuroscience*, *26*(37), 9494-9502.
van Moorselaar, D., Daneshtalab, N., & Slagter, H. A. (2021). Neural mechanisms underlying distractor inhibition on the basis of feature and/or spatial expectations. *Cortex*, *137*, 232-250.
van Moorselaar, D., Foster, J. J., Sutterer, D. W., Theeuwes, J., Olivers, C. N., & Awh, E. (2018). Spatially selective alpha oscillations reveal moment-by-moment trade-offs

between working memory and attention. *Journal of cognitive neuroscience*, *30*(2), 256-266.
van Moorselaar, D., Huang, C., & Theeuwes, J. (2023). Electrophysiological Indices of Distractor Processing in Visual Search Are Shaped by Target Expectations. *Journal of cognitive neuroscience*, *35*(6), 1032-1044.
van Moorselaar, D., Lampers, E., Cordesius, E., & Slagter, H. A. (2020). Neural mechanisms underlying expectation-dependent inhibition of distracting information. *Elife*, *9*, e61048.
van Moorselaar, D., & Slagter, H. A. (2019). Learning What Is Irrelevant or Relevant: Expectations Facilitate Distractor Inhibition and Target Facilitation through Distinct Neural Mechanisms. *Journal of Neuroscience*, *39*(35), 6953-6967. https://doi.org/https://doi.org/10.1523/JNEUROSCI.0593-19.2019
van Moorselaar, D., Theeuwes, J., & Van der Stigchel, S. (2025). Alpha-band activity tracks reflexive changes in the breadth of the zoom lens of attention. *Journal of Neuroscience*.
van Moorselaar, D., Wang, N., & Theeuwes, J. (2025). Differential Neural Mechanisms Underlying Inhibition of Color and Dynamic Motion Distractors. *Journal of cognitive neuroscience*, *37*(3), 543-554.
Woodman, G. F., & Luck, S. J. (2003). Serial deployment of attention during visual search. *Journal of Experimental Psychology: Human Perception and Performance*, *29*(1), 121.
Worden, M. S., Foxe, J. J., Wang, N., & Simpson, G. V. (2000). Anticipatory biasing of visuospatial attention indexed by retinotopically specific-band electroencephalography increases over occipital cortex. *J Neurosci*, *20*(RC63), 1-6.